

Operation Ensnaris

A deception-based cybersecurity project using the Cowrie Honeypot to capture attacker behaviour, analyse command activity and map observations to the MITRE ATT&CK framework.

Disclaimer: This project was conducted entirely within an authorised virtual lab environment. Any usernames, passwords, credentials, IP addresses, hostnames, commands, or log entries shown in this report are simulated for educational and defensive cybersecurity purposes. No real credentials, third-party systems, or unauthorised networks were used or targeted.

Executive Summary

Operation Ensnaris is a deception-based cybersecurity project that used the Cowrie Honeypot to capture and analyse attacker behaviour. The purpose of the project was to understand how deception can be used to improve threat detection, reduce low-value noise and produce useful telemetry that could later be mapped to the MITRE ATT&CK framework (MITRE ATT&CK, n.d.).

The project involved setting up Cowrie in a Ubuntu virtual machine, solving configuration and runtime issues and then using simulated SSH sessions to test whether the Honeypot was capturing useful interaction data. Once stable, the system recorded usernames, passwords, commands, session activity and timestamps, which gave me enough to start analysing attacker behaviour and command patterns (Cowrie Documentation, 2026a).

The project proved the concept at a basic but useful level. It showed that Cowrie can capture meaningful attacker interaction data, but also that a simple Honeypot still has limitations — especially around realism, manual log filtering and the amount of work still required to turn raw telemetry into higher-quality security alerts (Piskozub et al., 2023; Salah et al., 2022).

Table of Contents

1. Introduction
2. Project Objectives
3. Methodology
4. Implementation
5. Findings and Analysis
6. Reflection, Limitations and Improvements
7. Security Engineering Concepts
8. Conclusion
9. References

1. Introduction

Operation Ensnaris was developed as a deception-based security project using the Cowrie Honeypot to capture and analyse attacker behaviour. The main idea behind the project was that if I could deploy a believable enough Honeypot and get attacker interaction, I could then review the logs, identify patterns in the commands, and use that to better understand what the attacker was trying to do (Cowrie Project, n.d.).

The project involved designing and deploying Cowrie within a controlled virtual environment to simulate a vulnerable system and attract interaction through SSH. The system was intended to capture login attempts, executed commands and behavioural patterns, and then map that activity to the MITRE ATT&CK framework so the logs were not just raw output with no structure (MITRE ATT&CK, n.d.).

This report has been structured as a formal engineering report drawing from weekly logbook entries completed throughout the project. The wording has been kept as close as possible to the original entries while the content has been reorganised so the report reads as one connected piece of work rather than a week-by-week logbook.

2. Project Objectives

The main project goal was to develop a deception-based security tool, named Operation Ensnaris, using a Honeypot to capture and analyse attacker behaviour with the aim of understanding how deception can reduce SOC alert fatigue and improve threat detection (MITRE ATT&CK, n.d.).

The technical component of the project included:

- Deploying and configuring a working Cowrie Honeypot
- Collecting and analysing attack data
- Classifying attacker behaviour — for example, reconnaissance and privilege-related activity
- Mapping observed commands to the MITRE ATT&CK framework

This required a structured approach to both the technical deployment and the analytical interpretation of the logs captured throughout the project.

Another objective was to explore whether a lightweight AI-assisted classification layer could eventually be used to help group behaviour automatically. This was not fully implemented in the project but remained part of the future direction because it links back to the longer-term goal of reducing manual workload in a SOC environment (Cowrie Project, n.d.).

3. Methodology

The methodology for this project was practical and based around building, testing, collecting data and then analysing the output. I started by setting up a Ubuntu Virtual Machine on VirtualBox, installing Cowrie and configuring the Honeypot so it would run in a stable way and actually capture useful information (Cowrie Documentation, 2026b).

Once the system was working I ran multiple simulated SSH sessions using different usernames and passwords to see how well Cowrie captured login attempts and post-authentication behaviour. After that, I reviewed the logs, filtered the output and looked for repeated behaviour patterns in the commands that were executed.

The last part of the methodology was to interpret what I captured rather than just listing commands. That meant reviewing the logs, grouping commands into behaviour categories and linking observed actions to recognised cybersecurity concepts using the MITRE ATT&CK framework. This helped turn the Honeypot logs into something more useful for security analysis and supported the broader aim of reducing alert fatigue in SOC environments (MITRE ATT&CK, n.d.; Piskozub et al., 2023).

4. Implementation

4.1 Environment Setup

The implementation phase of Operation Ensnaris focused on building the environment, troubleshooting Cowrie and validating that it could collect meaningful attacker interaction data. The first step was setting up a Ubuntu Virtual Machine on VirtualBox, installing Cowrie and then configuring the Honeypot.

4.2 Configuration Issues and Debugging

Initial deployment attempts did not go as planned — it resulted in several configuration and runtime issues. These included failures during service startup, port connectivity issues and default configuration settings that were clashing with my devices and network settings. These issues required significantly more debugging time than I originally planned for (Cowrie Documentation, 2026b).

```
cowrie@ethan-VirtualBox:~/cowrie/cowrie-fresh$ ./bin/cowrie restart
cowrie is not running.
Starting cowrie: [twistd --umask=0022 --pidfile /home/cowrie/cowrie/cowrie-fresh/var/run/cowrie.pid --logger cowrie.pyt
hon.logfile.logger cowrie]...
Traceback (most recent call last):
  File "/home/cowrie/cowrie/cowrie-fresh/./bin/cowrie", line 6, in <module>
    sys.exit(run())
  File "/home/cowrie/cowrie/cowrie-fresh/src/cowrie/scripts/cowrie.py", line 276, in run
    main()
  File "/home/cowrie/cowrie/cowrie-fresh/src/cowrie/scripts/cowrie.py", line 263, in main
    cowrie_restart(parsed_args.args)
  File "/home/cowrie/cowrie/cowrie-fresh/src/cowrie/scripts/cowrie.py", line 221, in cowrie_restart
    cowrie_start(args)
  File "/home/cowrie/cowrie/cowrie-fresh/src/cowrie/scripts/cowrie.py", line 162, in cowrie_start
    os.execvp(twisted_args[0], twisted_args)
  File "/usr/lib/python3.10/os.py", line 575, in execvp
    _execvpe(file, args)
  File "/usr/lib/python3.10/os.py", line 617, in _execvpe
    raise last_exc
  File "/usr/lib/python3.10/os.py", line 608, in _execvpe
    exec_func(fullname, *argrest)
  File "/usr/lib/python3.10/os.py", line 608, in _execvpe
    exec_func(fullname, *argrest)
FileNotFoundError: [Errno 2] No such file or directory
cowrie@ethan-VirtualBox:~/cowrie/cowrie-fresh$
```

I had to spend a lot more time than I originally planned debugging these issues. The repeated "Unhandled Errors" eventually lead me to duplicated SSH configuration issues and multiple listen_endpoints within the Cowrie configuration file. To solve this I had to improve my command-line investigation techniques and navigation, which included targeting certain files with conflicting commands and in some cases having to write my own files because they were not included in the initial Cowrie install (Cowrie Documentation, 2026b).

```
(cowrie-env) cowrie@ethan-VirtualBox:~/cowrie/cowrie-fresh$ grep -n listen_endpoints etc/cowrie.cfg
221:listen_endpoints = tcp:6415:interface=127.0.0.1
544:listen_endpoints = tcp:2223:interface=0.0.0.0
638:# (default: listen_endpoints = tcp:2222:interface=0.0.0.0)
640:# listen_endpoints = systemd:domain=INET:index=0
641:# For both IPv4 and IPv6: listen_endpoints = tcp6:2222:interface=\\:
643:# e.g listen_endpoints = "tcp:2222:interface=0.0.0.0 tcp:1022:interface=0.0.0.0" will result listening both on ports 2222 and 10
22
646:listen_endpoints = tcp:2222:interface=0.0.0.0
719:# (default: listen_endpoints = tcp:2223:interface=0.0.0.0)
721:# listen_endpoints = systemd:domain=INET:index=0
722:# For IPv4 and IPv6: listen_endpoints = tcp6:2223:interface=\\: tcp:2223:interface=0.0.0.0
724:# e.g "listen_endpoints = tcp:2223:interface=0.0.0.0 tcp:2323:interface=0.0.0.0" will result listening both on ports 2223 and 23
23
727:listen_endpoints = tcp:2223:interface=0.0.0.0
```

Part of this also included targeting files with conflicting commands and, in some cases, having to write my own files as they were not included in the initial install. This was one of the points where I realised the project was going to be more than just installation, because getting the service to run properly depended on understanding the environment and the configuration in detail (Cowrie Documentation, 2026b).

4.3 Port Configuration

Normally Honeypots are installed on port 2222 as it is the default for popular SSH Honeypot software such as Cowrie, however within my household I could not use the default configuration because something else was already listening to that port. To resolve this the Honeypot listener was moved to port 2223, this was a practical fix that made the deployment stable enough to continue (Cowrie Documentation, 2026b).

4.4 Data Collection

Once Cowrie was stable on port 2223, I moved into data collection. I used multiple usernames during testing including ethan, admin, root and guest. Different passwords were also entered to simulate common attacker credential attempts. Once access was gained I executed commands such as whoami, ls, cat /etc/passwd, uname -a, netstat and ps aux (Cowrie Project, n.d.).

The Cowrie logs functioned and recorded usernames, passwords, commands, session activity and timestamps. This confirmed the Honeypot was functioning as intended and capable of recording useful data for MITRE ATT&CK mapping and behavioural analysis (Cowrie Project, n.d.; MITRE ATT&CK, n.d.).

Cowrie Logs

```
2026-04-20T16:17:50.384142Z [twisted.conch.ssh.sessionInfo] Getting shell
2026-04-20T16:18:00.203499Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: sudo su
2026-04-20T16:18:00.204534Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: sudo su
2026-04-20T16:18:17.268197Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: passwd
2026-04-20T16:18:17.270258Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: passwd
2026-04-20T16:18:20.116572Z [HoneyPotSSHTransport,1,127.0.0.1] INPUT (passwd): newpassword123
2026-04-20T16:18:34.034803Z [HoneyPotSSHTransport,1,127.0.0.1] INPUT (passwd): newpassword123
2026-04-20T16:18:39.294611Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: id
2026-04-20T16:18:39.296157Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: id
2026-04-20T16:19:08.053588Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: ssh admin@localhost -p 2223
2026-04-20T16:19:08.054812Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: ssh admin@localhost -p 2223
2026-04-20T16:19:13.115486Z [HoneyPotSSHTransport,1,127.0.0.1] INPUT (ssh): yes
2026-04-20T16:19:21.083702Z [HoneyPotSSHTransport,1,127.0.0.1] INPUT (ssh): Adminfakepassword
2026-04-20T16:19:46.331777Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: su admin
2026-04-20T16:19:46.332949Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: su admin
2026-04-20T16:20:01.476552Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: uname -a
2026-04-20T16:20:01.478134Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: uname -a
2026-04-20T16:20:10.365779Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: netstat
2026-04-20T16:20:18.367733Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: netstat
2026-04-20T16:20:19.507217Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: ps aux
2026-04-20T16:20:19.507833Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: ps aux
2026-04-20T16:21:21.647828Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: ssh guest@localhost -p 2223
2026-04-20T16:21:21.649312Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: ssh guest@localhost -p 2223
2026-04-20T16:21:23.828405Z [HoneyPotSSHTransport,1,127.0.0.1] INPUT (ssh): yes
2026-04-20T16:21:33.484302Z [HoneyPotSSHTransport,1,127.0.0.1] INPUT (ssh): Guestpasscapture
2026-04-20T16:22:07.376419Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: su guest
2026-04-20T16:22:07.376922Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: su guest
2026-04-20T16:22:10.208268Z [HoneyPotSSHTransport,1,127.0.0.1] CMD: ls
2026-04-20T16:22:10.208903Z [HoneyPotSSHTransport,1,127.0.0.1] Command found: ls
2026-04-20T16:22:50.332394Z [-] Timeout reached in HoneyPotSSHTransport
2026-04-20T16:22:50.334360Z [HoneyPotSSHTransport,1,127.0.0.1] Closing TTY Log: var/lib/cowrie/tty/a15de4b59b45cb289bfc8beaef43df15d7eb959ead9113c0eb after 299.9 seconds
2026-04-20T16:22:50.336137Z [cowrie.ssh.transport:HoneyPotSSHTransportInfo] connection lost
2026-04-20T16:22:50.337178Z [HoneyPotSSHTransport,1,127.0.0.1] Connection lost after 312.9 seconds
```

Simulated Attack

```
ethan
ethan@svr04:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Listing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101:/:/var/lib/libuuid:/bin/sh
sshd:x:101:65534:/:/run/ssh:/usr/sbin/nologin
phill:x:1000:1000:Phill California,,,:/home/phill:/bin/bash
ethan@svr04:~$ ls
ethan@svr04:~$ pwd
/home/ethan
ethan@svr04:~$ cd ..
ethan@svr04:/home$ ls
ethan phill
ethan@svr04:/home$ cd phill
ethan@svr04:/home/phill$ ls
ethan@svr04:/home/phill$ pwd
/home/phill
ethan@svr04:/home/phill$ ls
ethan@svr04:/home/phill$ exit
Connection to localhost closed.
```

5. Findings and Analysis

5.1 Reconnaissance Behaviour

After the implementation stage the focus moved into analysing the attacker interaction captured inside Cowrie. On review of the commands entered after gaining shell access, most of the activity appeared reconnaissance based. Common examples included whoami, ls, id, uname -a, netstat and ps aux. These commands would lead me to believe the attacker is trying to understand the environment, current privileges, running processes and active network connections (MITRE ATT&CK, n.d.; Netscout, n.d.).

```
cowrie-env) cowrie@ethan-VirtualBox:~/cowrie/cowrie-fresh$ grep "eventId":"cowrie.command.input" var/log/cowrie/cowrie.json | tail
-10
{"eventId":"cowrie.command.input","input":"passwd","message":"CMD: passwd","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f1-9b
a-11ebb78baf20","timestamp":"2026-04-20T16:18:17.268197Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"id","message":"CMD: id","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f1-9bea-11ebb
8baf20","timestamp":"2026-04-20T16:18:39.294611Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"ssh admin@localhost -p 2223","message":"CMD: ssh admin@localhost -p 2223","sensor":"ethan
VirtualBox","uuid":"077373d6-34a0-11f1-9bea-11ebb78baf20","timestamp":"2026-04-20T16:19:08.053588Z","src_ip":"127.0.0.1","session":"
11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"su admin","message":"CMD: su admin","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f
-9bea-11ebb78baf20","timestamp":"2026-04-20T16:19:46.331777Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"uname -a","message":"CMD: uname -a","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f
-9bea-11ebb78baf20","timestamp":"2026-04-20T16:20:01.476552Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"netstat","message":"CMD: netstat","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f1-
bea-11ebb78baf20","timestamp":"2026-04-20T16:20:10.365779Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"ps aux","message":"CMD: ps aux","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f1-9b
a-11ebb78baf20","timestamp":"2026-04-20T16:20:19.507217Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"ssh guest@localhost -p 2223","message":"CMD: ssh guest@localhost -p 2223","sensor":"ethan
VirtualBox","uuid":"077373d6-34a0-11f1-9bea-11ebb78baf20","timestamp":"2026-04-20T16:21:21.647828Z","src_ip":"127.0.0.1","session":"
11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"su guest","message":"CMD: su guest","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f
-9bea-11ebb78baf20","timestamp":"2026-04-20T16:22:07.376419Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
{"eventId":"cowrie.command.input","input":"ls","message":"CMD: ls","sensor":"ethan-VirtualBox","uuid":"077373d6-34a0-11f1-9bea-11ebb
8baf20","timestamp":"2026-04-20T16:22:10.208268Z","src_ip":"127.0.0.1","session":"11eaa6c4af75","protocol":"ssh"}
cowrie-env) cowrie@ethan-VirtualBox:~/cowrie/cowrie-fresh$
```

5.2 Privilege-Related Activity

When I then simulated commands such as su admin, su guest and passwd they indicated attempts to change the user context, manipulate credentials and move into more privileged activity. This showed a progression from initial reconnaissance into more privileged behaviour, even though the environment was still controlled and sessions were simulated (MITRE ATT&CK, n.d.). Within the simulation, I had moved from the Pre-attack stage to the Exploit and Control stages.

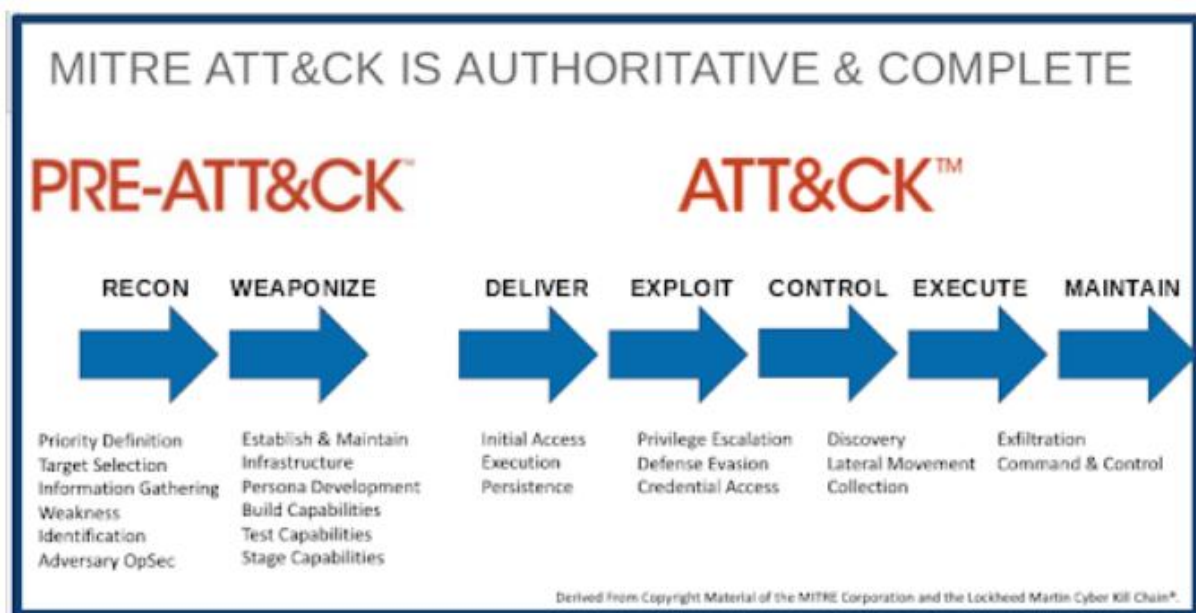
5.3 MITRE ATT&CK Mapping

A major point that stood out to me was that raw logs became more valuable when they were grouped into behaviour categories instead of being treated as isolated lines. Large log files initially appeared noisy and I needed to apply filtering before they became useful for practical analysis (Piskozub et al., 2023).

This was also where the MITRE ATT&CK framework became useful. The observed activity could be linked to account discovery, process discovery, system discovery and other forms of post-access behaviour. This supported the longer-term objective of the project which was to turn raw Honeypot logs into higher-quality security alerts and reduce SOC alert fatigue by giving more structure to the output (MITRE ATT&CK, n.d.).

Table 1: Observed Commands Mapped to MITRE ATT&CK Framework

Observed Command	What it Suggested to Me	ATT&CK Alignment
whoami	Checking current user context	System Owner/User Discovery (MITRE ATT&CK, n.d.)
id	Checking identity and privilege details	Account Discovery (MITRE ATT&CK, n.d.)
uname -a	Learning more about the host system	Discovery-related behaviour (MITRE ATT&CK, n.d.)
netstat	Looking at active network connections	System Network Connections Discovery (Netscout, n.d.)
ps aux	Looking at running processes	Process Discovery (MITRE ATT&CK, n.d.)
su admin / su guest	Trying to change user context	Account and privilege-related behaviour (MITRE ATT&CK, n.d.)
passwd	Attempting a credential-related action	Credential manipulation behaviour (MITRE ATT&CK, n.d.)



From a SOC perspective the captured commands were relevant because actions such as user switching, password changes, process discovery and network inspection would likely justify analyst attention. That does not mean the prototype solved alert fatigue by itself, but it did show that deception can produce telemetry that is more meaningful when the logs are filtered and interpreted properly (Piskozub et al., 2023).

5.4 Environmental Realism

The findings also showed a limitation in the realism of the environment. From an attacker perspective the simulated shell functioned correctly but appeared limited in realism. The environment felt relatively empty and lacked higher-value bait assets such as sensitive-looking folders or files. I had placed a fake file in there called "Finance26", however within my simulation I ended up directing myself into the contents of the fake file (so the file name was never seen). This suggests that while Cowrie was effective for capturing basic attacker behaviour, additional deception content would likely improve engagement and encourage deeper interaction (Piskozub et al., 2023; Salah et al., 2022).

6. Reflection, Limitations and Improvements

6.1 Project Reflection

The reflection stage focused on consolidating the findings, documenting prototype limitations and thinking about where the system could go from here. The main objective of Operation Ensnaris was to track a Bad Actor's behaviour, understand their actions and implement the MITRE ATT&CK framework in order to better defend against the adversary. At a basic level the project achieved that objective and validated the use of a deceptive tool (MITRE ATT&CK, n.d.).

The key strength that stood out to me was that Cowrie successfully captures meaningful Bad Actors' behavioural data, as seen within the simulations and evidence from prior weeks. The Cowrie logs recorded inputs such as usernames, passwords, session activity and commands such as ps aux, su, netstat and passwd. When this data is compiled it provides relevant insight into attacker intent, reconnaissance methods and attempts to escalate privileges (Cowrie Project, n.d.).

6.2 Limitations

However, upon reflection of the project there were several weaknesses. The main three I identified were that HoneyPot lacked realism, the current workflow relies heavily on manual log review and filtering which could become a flaw when scaling to a SOC environment, and the whole dataset was created and tested by me. This means I already knew my intent and the limitations of my ability, so the data is useful for a proof of concept but I can not overstate it beyond that. From spending plenty of time on the creation of the HoneyPot I was able to navigate very easily thought out the whole honeypot, a "Normal" attacker would not be able to do so with such ease. I should have done the simulation as a attacker would have done so.

When I am looking at evasion techniques, the software is without flaws. An attacker may notice the file suite is not a real environment due to how unrealistic it appears, and it would then be simple for them to exit and move on. The Bad Actor leaving the environment would result in Cowrie not capturing the relevant information and behavioural data from the attack which is arguably one of the main objectives of the software. To improve this I would need to build out the files further and input additional data that is fake, or at minimum not confidential (Piskozub et al., 2023; Salah et al., 2022).

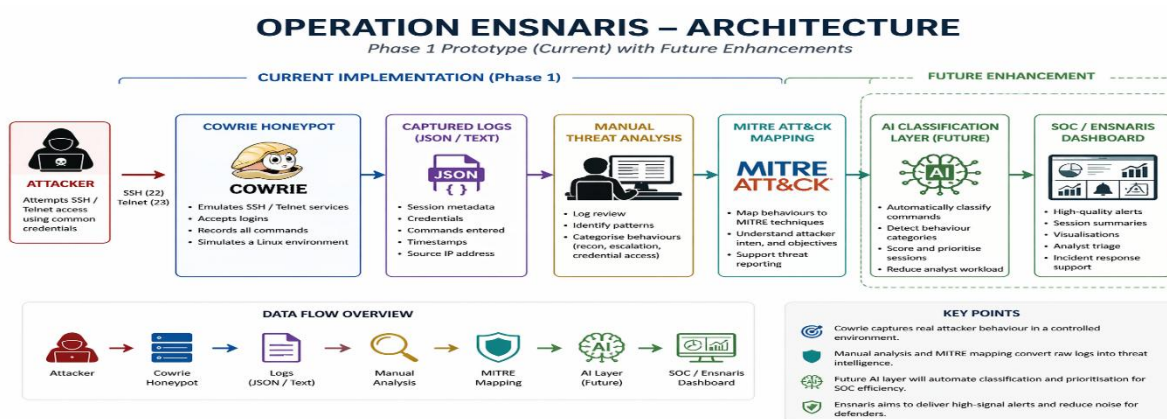
6.3 Future Improvements

There are still clear improvements that could be made. These include:

- Automated log parsing to reduce manual review time
- More structured MITRE ATT&CK mapping
- AI-assisted behaviour classification to group commands automatically
- A dashboard for SOC analysts to visualise the data

That future-state design aligns more closely with the original project goal of reducing alert fatigue through smarter detection (Cowrie Project, n.d.; MITRE ATT&CK, n.d.).

In summary the prototype of Operation Ensnaris successfully proved the concept while also highlighting generic issues with Cowrie when it is not personalised enough. Overall it was useful to simulate SOC analyst activity and inspect the logs myself rather than only discussing the idea in theory.



7. Security Engineering Concepts

One of the main security engineering concepts in this project is deception. Instead of only trying to block or prevent attacks, deception creates a controlled false target so that attacker behaviour can be observed and analysed. Research on Honeypots and cyber deception supports the idea that deception can improve defender visibility and produce useful telemetry when it is designed and monitored properly (Piskozub et al., 2023; Salah et al., 2022).

Another important concept is signal versus noise. One of the repeated issues throughout the project was that raw logs do not automatically become useful alerts. I had to filter and group the data before it became meaningful, which links closely to the problem of alert fatigue in SOC environments (Cowrie Project, n.d.).

The project also connects well to behaviour-based analysis. By taking common Linux commands and linking them to the MITRE ATT&CK framework, the project moved from simple command capture into a more structured way of analysing attacker behaviour. This made the evidence more useful because it connected technical activity to recognised tactics and techniques (MITRE ATT&CK, n.d.; Netscout, n.d.).

Finally the project reinforced that secure engineering is not just about choosing a good tool. The Cowrie deployment only became useful after the configuration issues, duplicate entries and environmental conflicts were resolved. That showed me that a security control has to be operational and stable before it can actually provide value (Cowrie Documentation, 2026b).

8. Conclusion

Operation Ensnaris as my capstone project I successfully developed a working deception-based cybersecurity prototype using the Cowrie Honeypot. The project captured attacker interaction data, analysed command behaviour and showed that the observed activity could be mapped to the MITRE ATT&CK framework in a way that supported useful behavioural interpretation (Cowrie Project, n.d.; MITRE ATT&CK, n.d.).

One of the strongest parts of the project was that it did not stop at deployment. It moved from setup problems and debugging into actual evidence collection and then into analysis. Commands such as `whoami`, `id`, `uname -a`, `netstat`, `ps aux`, `su` and `passwd` showed that even simple shell activity can reveal a lot about what an attacker is trying to do inside a system (MITRE ATT&CK, n.d.; Netscout, n.d.).

At the same time I was realistic about the project's weaknesses. The environment was basic, the logs required manual filtering and the simulated dataset has limitations. Even with those issues the prototype still demonstrated the value of deception as a security engineering concept and provided a solid base for future improvement (Piskozub et al., 2023; Salah et al., 2022).

In summary, Operation Ensnaris proved the concept successfully. It showed that Cowrie can be used not only to log attacker activity but also to support behavioural analysis, ATT&CK mapping and future ideas around smarter alerting and reduced analyst fatigue (Cowrie Documentation, 2026a; Cowrie Project, n.d.).

9. References

- Cowrie Documentation. (2026a). *Cowrie documentation*.
<https://docs.cowrie.org/en/latest/README.html>
- Cowrie Documentation. (2026b). *Installing Cowrie in seven steps*.
<https://docs.cowrie.org/en/latest/INSTALL.html>
- Cowrie Project. (n.d.). *Cowrie SSH/Telnet Honeypot*. GitHub.
<https://github.com/cowrie/cowrie>
- MITRE ATT&CK. (n.d.). *MITRE ATT&CK*. <https://attack.mitre.org>
- Netscout. (n.d.). *What is MITRE ATT&CK Discovery (TA0007)?*
<https://www.netscout.com/what-is-mitre-attack/discovery>
- Piskozub, A., Kochan, M., & Drobik, A. (2023). Honeypot and cyber deception as a tool for detecting cyber attacks. *CEUR Workshop Proceedings*. <https://ceur-ws.org/Vol-3374/paper06.pdf>
- Salah, K., Khan, M. A., & Alfalasi, A. (2022). *An analysis of honeypots and their impact as a cyber deception technology*. arXiv.
<https://arxiv.org/abs/2301.00045>